Cybersecurity and Threat Detection System

Implementation Summary Document

Module 1: User Authentication

Requirement: Secure access and role-based controls.

How We Built It:
- Used Python (Flask) alongside a local SQLite database (managed by Flask-SQLAlchemy) to store user credentials.
- Passwords are not stored in plain text; utilized the `bcrypt` library to securely hash all passwords.
- Implemented JSON Web Tokens (JWT) using `PyJWT`. When a user logs in via the `/api/auth/login` endpoint, they receive a token that must be attached to all future API requests.
- Implemented Role-Based Access Control (RBAC) with three tiers: Admin, Analyst, and Viewer. Certain endpoints (like User Management) are locked behind an `@admin_required` decorator.

Module 2: Log Collection

Requirement: Centralized ingestion of data from various sources (web servers, firewalls, etc.).

How We Built It:
- Created a `/api/logs` REST API endpoint in the Flask application (`app.py`) to receive incoming log data.
- All logs are normalized and stored in the `logs_normalised` table in the SQLite database, tracking the timestamp, source IP, event type, severity, and the raw message.
- To simulate a live network, built a standalone background Python script (`log_generator.py`). This script runs continuously, randomly generating realistic mock logs for Web Servers (HTTP requests), Firewalls (Connections allowed/blocked), and System Auth (logins), and sends them to the API.

Module 3: Threat Detection

Requirement: Signature-based and anomaly-based techniques to identify malicious activity.

How We Built It:
- Developed a dedicated background daemon called `detection_engine.py` that runs alongside the web server.
- The engine uses a configuration file (`rules.yaml`) that defines specific threat signatures. For example, a 'Brute Force' rule triggers if there are 5 AUTH_FAILURE events from the same IP within 300 seconds.
- Every 10 seconds, the detection engine queries the database for recent logs and evaluates them against these YAML rules. If a threshold is met, it registers a Threat in the database.

Module 4: Alert and Notification

Requirement: Dispatch alerts based on threat severity.

Cybersecurity and Threat Detection System

Implementation Summary Document

How We Built It:
- When the Threat Detection Engine confirms a threat, it generates an internal Alert record.
- Integrated WebSockets into the backend using `Flask-SocketIO`. The moment an alert is created, the backend emits a real-time `new_alert` event to any connected web browsers.
- For high-priority (CRITICAL or HIGH) threats, the engine executes simulated logic (printing to the terminal) to represent dispatching emergency Emails and SMS messages to Administrators.

Module 5: Reporting and Dashboard

Requirement: Centralized, real-time visual interface and automated reporting features.

How We Built It:
- Built a custom, responsive frontend using plain HTML, pure Vanilla CSS (no heavy frameworks), and Vanilla JavaScript.
- Designed a modern, dark-themed UI (`style.css`) with glassmorphism touches and smooth CSS micro-animations.
- Utilized the `Chart.js` library to render the interactive 'Threat Timeline (7 Days)' graph.
- The frontend connects to the WebSocket server to receive live data. When a log is ingested or an alert is generated, the UI updates instantly-populating the 'Live Activity Feed' and popping up toast notifications without requiring a page refresh.
- Built out the supplementary pages: a Threat history table, a searchable Log Explorer, and a Settings panel for Admins to view detection rules and manage user accounts.